

Package ‘countryriskindex’

March 20, 2026

Title Country Health Risk Calculation

Version 0.1.1

Description countryriskindex is an R package for computing the Country Health Risk Index (CHRI), a composite indicator designed to assess national health risks by integrating multiple dimensions of public health vulnerability.

License file LICENSE

Imports dplyr,
factoextra,
fmsb,
fs,
ggdendro,
ggfortify,
ggplot2,
ggradar (>= 0.2),
ggtext,
irr,
janitor,
leaflet,
plotly,
purrr,
quarto,
RColorBrewer,
readr,
readxl,
reshape2,
rlang,
rnaturalearth,
rnaturalearthdata,
scales,
sf,
shiny,
skimr,
stringi,
stringr,

tibble,
tidyr,
tidyselect,
viridis

Suggests checkmate,
covr,
crew,
devtools,
distill,
here,
htmltools,
knitr,
lintr,
lubridate,
qs,
qs2,
rstudioapi,
spelling,
tarchetypes,
targets,
testthat,
usethis,
withr

Remotes ricardo-bion/ggradar

Config/testthat/edition 3

Encoding UTF-8

Language en-US

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.3

Contents

add_capacity_score	4
add_hazard_score	4
add_localised_risk	5
add_overall_risk	6
add_proximity_score	6
add_severity	7
add_vulnerability_score	8
createimg_bars	9
createimg_cap_bars	9
createimg_daly_bars	10
createimg_risk_bars	10
createimg_risk_hist	11
createimg_u5mr_bars	11
createimg_vul_bars	12

createmap	12
createmap_adjusted_risk_score	13
createmap_cap_avg	14
createmap_cap_score	14
createmap_DALY_avg	15
createmap_DALY_score	15
createmap_risk_score	16
createmap_severity_score	16
createmap_u5mr_avg	17
createmap_u5mr_did	17
createmap_vul_score	18
create_3d_plot	18
create_correlation_matrix_with_groups	19
create_disease_plot_from_long	20
create_disease_scatter	20
create_lollipop	21
create_lollipop_plot_comp	22
create_lollipop_plot_shift	23
create_outbreak_concern	23
create_question_plot_from_long	24
export_gpkg	25
extract_radar_data	26
geometric_mean	26
import_crpi_excel	27
invert_0_10	27
invert_0_100	28
merge_health_datasets	28
normalise_quantiles	29
null	29
pivot_disease_data	30
plot_all_diseases_from_long	31
plot_all_questions_from_long	32
preprocess_crpi	32
preprocess_gbd_rates_by_cause	33
preprocess_haq_index	34
preprocess_inform	34
preprocess_proximity_assessment	35
preprocess_severity	36
preprocess_survey	36
preprocess_u5mr	37
preprocess_who_data	38
process_ami_survey	39
run_app	40
summarise_outbreak_data	40
update_risk_with_cm	41
update_score	42

add_capacity_score	Compute Health System Capacity Score
Description	
Calculates a composite health system capacity score using four components:	
• Governance: Inverted gov_effectivess (scaled from 0–10 to 0–100). • Financing: Average of uhc_coverage and inverted health_per_capita (scaled to 0–100). • Resources: doctor_density. • Services: haqi.	
Usage	
add_capacity_score(df)	
Arguments	
df	A data frame with the required columns for capacity computation.
Details	
The final score is the normalized geometric mean of the four components.	
Value	
The input data frame with added columns:	
• governance, financing, resources, services • capacity_score — normalized composite score.	
add_hazard_score	Compute Hazard Score from GBD Rates

Description	
Calculates a normalized hazard score based on the "All causes" DALY rate from GBD data. The score is computed by dividing each value by the maximum observed value.	
Usage	
add_hazard_score(df)	
Arguments	
df	A data frame containing a column named "All causes".

Value

The input data frame with an added column hazard_score.

add_localised_risk	<i>Add localised risk to country-level risk data</i>
--------------------	--

Description

This function merges proximity adjustment scores with a country-level risk table and computes a new column localised_risk using an odds-based adjustment.

Usage

```
add_localised_risk(df, proximity_adjustments)
```

Arguments

df	A tibble with one row per country, including ISO_A3 and severity_adjusted_risk
proximity_adjustments	A tibble with columns: ISO_A3, disease_score, outbreak_score, people_score, final_score

Value

A tibble with all original columns plus localised_risk

Examples

```
df <- tibble::tibble(
  ISO_A3 = c("NGA", "TUV", "UGA"),
  severity_adjusted_risk = c(0.5, 0.4, 0.6)
)

proximity_adjustments <- tibble::tibble(
  ISO_A3 = c("NGA", "TUV", "UGA"),
  disease_score = c(0.3, 0.3, 0.3),
  outbreak_score = c(0.0892, 0.0892, 0.0892),
  people_score = c(0.301, 0.301, 0.301),
  final_score = c(0.690, 0.690, 0.690)
)

add_localised_risk(df, proximity_adjustments)
```

add_overall_risk	<i>Compute Overall Risk Score</i>
------------------	-----------------------------------

Description

Calculates the overall risk score as the geometric mean of:

- hazard_score
- vulnerability_score
- capacity_score

Usage

```
add_overall_risk(df)
```

Arguments

df	A data frame containing the three component scores.
----	---

Details

This score reflects the combined impact of hazard, vulnerability, and system capacity.

Value

The input data frame with an added column overall_risk.

add_proximity_score	<i>Add proximity adjustment risk scores from a reporting form</i>
---------------------	---

Description

Calculates risk adjustment scores for each row in a reporting form based on:

- Disease-specific delay responses (diagnosis and transport)
- Outbreak concern responses
- Population-based score using log-transformed local residents, expatriates, and families

The final score is the mean of:

- Disease-specific score (± 0.1 per matched delay question)
- Outbreak concern score (weighted by concern level and outbreak proportions)
- People concern score: log10-transformed population values adjusted and summed

Usage

```
add_proximity_score(long, form, threshold = 0.5)
```

Arguments

`long` A tibble: disease, question, response, n, prop

`form` A tibble: reporting form rows

`threshold` Numeric, threshold for disease scoring (default 0.5)

Value

Tibble with: ISO_A3, disease_score, outbreak_score, people_score, final_score

add_severity	<i>Compute Severity-Adjusted Risk Score</i>
--------------	---

Description

Adjusts the overall risk score using the INFORM Severity Index. The adjustment formula is:

$$overall_risk + ((severity_index - 1)/4) * (1 - overall_risk)$$

This increases the risk score proportionally to the severity index (range 1–5).

Usage

```
add_severity(df)
```

Arguments

`df` A data frame with columns `overall_risk` and `severity_index`.

Value

The input data frame with an added column `severity_adjusted_risk`.

`add_vulnerability_score`*Compute Composite Vulnerability Score*

Description

Calculates a composite vulnerability score using indicators from WHO and INFORM datasets. The score is based on four conceptual components:

- **Socioeconomic vulnerability:** `soc_econ_vulnerability`, inverted on a 0–10 scale.
- **Infrastructure:** Computed as the geometric mean of:
 - `comms` — average of connectivity, electricity, internet, mobile
 - `physical` — average of road_density, improved_water, improved_sanitation

Both components are inverted on a 0–10 scale before aggregation.

- **Vulnerable groups:** Geometric mean of `uprooted` and `food_sec`, inverted on a 0–10 scale.
- **Education:** `adult_literacy`, inverted on a 0–10 scale.

Usage

```
add_vulnerability_score(df)
```

Arguments

<code>df</code>	A data frame with the required columns for vulnerability computation.
-----------------	---

Details

The final vulnerability score is the normalized geometric mean of the four components.

Value

The input data frame with added columns:

- `vulnerable_groups`, `comms`, `physical`, `infrastructure`
- `vulnerability_score` — normalized composite score.

createimg_bars	<i>Create a Flexible Horizontal Bar and Point Plot</i>
----------------	--

Description

Generates a horizontal bar/point plot using ggplot2, with optional lineranges for uncertainty intervals. Supports tidy evaluation for flexible column input and reordering of the y-axis.

Usage

```
createimg_bars(
  data,
  x_var,
  y_var,
  color_var,
  x_label,
  y_label,
  linerange = NULL
)
```

Arguments

data	A data frame containing the plotting data.
x_var	Unquoted column name for the x-axis.
y_var	Unquoted column name for the y-axis (used for reordering).
color_var	Unquoted column name for coloring the points.
x_label	Label for the x-axis.
y_label	Label for the y-axis.
linerange	Optional list with xmin and xmax as symbols for linerange plotting.

Value

A ggplot object representing the horizontal bar/point plot.

createimg_cap_bars	<i>Lack of Coping Capacity plot</i>
--------------------	-------------------------------------

Description

Lack of Coping Capacity plot

Usage

```
createimg_cap_bars(risk_score = NULL)
```

Arguments

risk_score Data with capacity values

Value

A ggplot object

createimg_daly_bars	<i>DALY Rate plot</i>
---------------------	-----------------------

Description

DALY Rate plot

Usage

createimg_daly_bars(gbd_rates = NULL)

Arguments

gbd_rates Data with DALY values

Value

A ggplot object

createimg_risk_bars	<i>Risk Score plot</i>
---------------------	------------------------

Description

Risk Score plot

Usage

createimg_risk_bars(risk_score = NULL)

Arguments

risk_score Data with risk values

Value

A ggplot object

createimg_risk_hist	<i>Create a Histogram of Health Risk Scores</i>
---------------------	---

Description

Generates a histogram of the overall health risk scores using ggplot2. The histogram is colored by the 'risk' variable and hides the legend.

Usage

```
createimg_risk_hist(risk_score = NULL)
```

Arguments

risk_score A data frame containing a column named 'overall_risk' and optionally 'risk'.

Value

A ggplot2 histogram object.

createimg_u5mrBars	<i>Under-5 Mortality Rate plot</i>
--------------------	------------------------------------

Description

Under-5 Mortality Rate plot

Usage

```
createimg_u5mrBars(u5_mr = NULL)
```

Arguments

u5_mr Data with U5MR values

Value

A ggplot object

createimg_vul_bars	<i>Vulnerability plot</i>
--------------------	---------------------------

Description

Vulnerability plot

Usage

```
createimg_vul_bars(risk_score = NULL)
```

Arguments

risk_score	Data with vulnerability values
------------	--------------------------------

Value

A ggplot object

createmap	<i>Generate a Flexible Interactive Choropleth Map</i>
-----------	---

Description

Creates an interactive choropleth map using leaflet, allowing dynamic specification of the data source, value column, color palette, and legend. It supports optional zooming to a specific country and is designed to generalize mapping logic across multiple datasets.

Usage

```
createmap(  
  data = NULL,  
  join_key = "iso_a3",  
  data_key = "REF_AREA",  
  value_column = "value",  
  palette = "YlOrRd",  
  legend_title = "Legend",  
  label_prefix = "Value:",  
  zoom_to_country = NULL  
)
```

Arguments

data	A data frame containing the values to be mapped. Must include a country code column.
join_key	Column name in the world map data to join on (default is "iso_a3").
data_key	Column name in data to join with join_key.
value_column	Column in data containing the values to visualize.
palette	Name of the color palette to use (e.g., "YlOrRd", "viridis").
legend_title	Title for the map legend.
label_prefix	Prefix for the tooltip label (default is "Value:").
zoom_to_country	Optional country name to zoom into on the map.

Value

A leaflet map object.

createmap_adjusted_risk_score

Interactive Map of Severity-adjusted Health Risk Score

Description

Displays a choropleth map of severity-adjusted health risk scores using a red-yellow-green color scale. Optionally zooms to a selected country.

Usage

```
createmap_adjusted_risk_score(db = NULL, zoom_to_country = NULL)
```

Arguments

db	A data frame with a "severity_adjusted_risk" column and ISO_A3-coded countries.
zoom_to_country	Optional country name to zoom into.

Value

A leaflet map object.

createmap_cap_avg	<i>Interactive Map of Lack of Coping Capacity</i>
-------------------	---

Description

Displays a choropleth map of INFORM coping capacity scores using a green-blue color scale.

Usage

```
createmap_cap_avg(inform_cap = NULL)
```

Arguments

`inform_cap` A data frame with a "capacity" column and ISO3-coded countries.

Value

A leaflet map object.

createmap_cap_score	<i>Interactive Map of Coping Capacity Score</i>
---------------------	---

Description

Displays a choropleth map of normalized coping capacity scores using a green-blue color scale.

Usage

```
createmap_cap_score(risk_score = NULL)
```

Arguments

`risk_score` A data frame with a "capacity_score" column and ISO3-coded countries.

Value

A leaflet map object.

createmap_DALY_avg	<i>Interactive Map of DALY Rate</i>
--------------------	-------------------------------------

Description

Displays a choropleth map of DALY rates for all causes using a blue color scale.

Usage

```
createmap_DALY_avg(gbd_rates = NULL)
```

Arguments

gbd_rates	A data frame with an "All causes" column and ISO_A3-coded countries.
-----------	--

Value

A leaflet map object.

createmap_DALY_score	<i>Interactive Map of DALY Quantiles</i>
----------------------	--

Description

Displays a choropleth map of DALY quantile scores using a plasma color scale.

Usage

```
createmap_DALY_score(gbd_rates = NULL)
```

Arguments

gbd_rates	A data frame with an "All causes" column and ISO_A3-coded countries.
-----------	--

Value

A leaflet map object.

createmap_risk_score *Interactive Map of Country Health Risk Score*

Description

Displays a choropleth map of overall health risk scores using a red-yellow-green color scale. Optionally zooms to a selected country.

Usage

```
createmap_risk_score(db = NULL, zoom_to_country = NULL)
```

Arguments

db A data frame with an "overall_risk" column and ISO_A3-coded countries.
zoom_to_country Optional country name to zoom into.

Value

A leaflet map object.

createmap_severity_score
 Interactive Map of Crisis Severity Index

Description

Displays a choropleth map of INFORM Severity Index scores using an accent color scale.

Usage

```
createmap_severity_score(db = NULL)
```

Arguments

db A data frame with a "severity_index" column and ISO_A3-coded countries.

Value

A leaflet map object.

createmap_u5mr_avg	<i>Interactive Map of Under-5 Mortality Rate</i>
--------------------	--

Description

Displays a choropleth map of average under-5 mortality rates by country using a yellow-red color scale.

Usage

```
createmap_u5mr_avg(u5_mr = NULL)
```

Arguments

u5_mr	A data frame with a "Total" column and ISO_A3-coded countries.
-------	--

Value

A leaflet map object.

createmap_u5mr_did	<i>Interactive Map of U5MR Intra-country Differences</i>
--------------------	--

Description

Displays a choropleth map of intra-country differences in under-5 mortality rates using a viridis color scale.

Usage

```
createmap_u5mr_did(u5_mr = NULL)
```

Arguments

u5_mr	A data frame with a "did" column and ISO_A3-coded countries.
-------	--

Value

A leaflet map object.

createmap_vul_score	<i>Interactive Map of Vulnerability Score</i>
---------------------	---

Description

Displays a choropleth map of normalized vulnerability scores using a viridis color scale.

Usage

```
createmap_vul_score(risk_score = NULL)
```

Arguments

risk_score	A data frame with a "vulnerability_score" column and ISO3-coded countries.
------------	--

Value

A leaflet map object.

create_3d_plot	<i>Create an Interactive 3D Scatterplot of Risk Components</i>
----------------	--

Description

Generates a 3D scatterplot using plotly to visualize three numeric dimensions (e.g., capacity, vulnerability, hazard) and color points by a fourth variable.

Usage

```
create_3d_plot(
  db,
  x_var,
  y_var,
  z_var,
  color_var,
  label_var,
  labels = c("X", "Y", "Z")
)
```

Arguments

db	A data frame containing the data to plot.
x_var	Name of the column to use for the x-axis (string).
y_var	Name of the column to use for the y-axis (string).
z_var	Name of the column to use for the z-axis (string).
color_var	Name of the column to use for coloring the points (string).
label_var	Name of the column to use for hover labels (string).
labels	A character vector of length 3 with axis labels: c(x_label, y_label, z_label).

Value

A plotly 3D scatterplot object.

`create_correlation_matrix_with_groups`*Correlation Matrix with Group Highlights*

Description

Computes and visualizes a Pearson correlation matrix for selected variables, with colored boxes highlighting conceptual groups (e.g., Health, Vulnerability).

Usage

```
create_correlation_matrix_with_groups(df)
```

Arguments

df	A data frame containing the variables to correlate.
----	---

Value

A ggplot2 heatmap of the correlation matrix with group annotations.

`create_disease_plot_from_long`*Create a stacked bar chart for a single disease using long-format data*

Description

Create a stacked bar chart for a single disease using long-format data

Usage

```
create_disease_plot_from_long(  
  long,  
  disease_name,  
  concern_levels = c("Very unconcerned", "Somewhat unconcerned",  
    "Neither concerned nor unconcerned", "Somewhat concerned", "Very concerned")  
)
```

Arguments

`long` The full long-format tibble from `pivot_disease_data()`.
`disease_name` Name of the disease to plot.
`concern_levels` Ordered vector of concern levels.

Value

A ggplot object.

`create_disease_scatter`*Create a 3D scatterplot of concern levels by disease*

Description

This function filters a tidyverse-style dataset to extract, for each disease, the proportion of "Very concerned" responses to three specific questions: "4h diagnosis delay", "4h transport delay", and "Patient life impact". It returns an interactive 3D scatterplot created with plotly.

Usage

```
create_disease_scatter(data)
```

Arguments

`data` A tibble with columns: disease, question, response, n, prop

Value

A plotly object containing the 3D scatterplot

Examples

```
# Example data for Acute Abdominal Pain
ong <- tibble::tibble(
  disease = rep("Acute Abdominal Pain", 3),
  question = c(
    "4h diagnosis delay",
    "4h transport delay",
    "Patient wellbeing impact"
  ),
  response = rep("Very concerned", 3),
  n = c(25, 40, 46),
  prop = c(0.3521, 0.5634, 0.6479)
)

create_disease_scatter(ong)
```

create_lollipop

Create a Lollipop Plot Comparing Two Numeric Columns

Description

Generates a lollipop plot comparing two numeric columns for each country. Optionally sorts countries by the absolute difference between the two values.

Usage

```
create_lollipop(
  df,
  country_col,
  x_col,
  y_col,
  title,
  sort_by_difference = TRUE,
  color_x = "blue",
  color_y = "red"
)
```

Arguments

df	A data frame containing the data to plot.
country_col	Name of the column with country names (string).
x_col	Name of the first numeric column (string).
y_col	Name of the second numeric column (string).

title	Title of the plot.
sort_by_difference	Logical. If TRUE, sorts countries by descending difference.
color_x	Color for the x_col points.
color_y	Color for the y_col points.

Value

A ggplot2 lollipop plot.

```
create_lollipop_plot_comp
```

Lollipop Plot Comparing Risk Score with CRPI Index

Description

Joins a risk score dataset with a CRPI dataset and creates a lollipop plot comparing the specified risk score column with the CRPI index.

Usage

```
create_lollipop_plot_comp(
  risk_score,
  crpi,
  x_col = "overall_risk",
  y_col = "Index",
  sort_by_difference = FALSE,
  color_x = "lightblue",
  color_y = "lightpink"
)
```

Arguments

risk_score	A data frame with ISO_A3, country, and a risk score column.
crpi	A data frame with iso_a3 and CRPI index columns.
x_col	Name of the risk score column in risk_score.
y_col	Name of the CRPI index column in crpi.
sort_by_difference	Logical. Whether to sort by absolute difference.
color_x	Color for the x_col points.
color_y	Color for the y_col points.

Value

A ggplot2 lollipop plot.

`create_lollipop_plot_shift`*Lollipop Plot Comparing Overall Risk and Severity-Adjusted Risk*

Description

Creates a lollipop plot comparing overall risk and severity-adjusted risk scores for each country.

Usage

```
create_lollipop_plot_shift(  
  risk_score,  
  sort_by_difference = FALSE,  
  color_x = "lightblue",  
  color_y = "deepskyblue"  
)
```

Arguments

<code>risk_score</code>	A data frame with columns ISO_A3, country, overall_risk, and severity_adjusted_risk.
<code>sort_by_difference</code>	Logical. Whether to sort by difference.
<code>color_x</code>	Color for overall_risk points.
<code>color_y</code>	Color for severity-adjusted risk points.

Value

A ggplot2 lollipop plot.

`create_outbreak_concern`*Plot a single stacked bar chart for outbreak concern questions*

Description

Visualizes concern levels across all outbreak-related questions in one plot. Each question is a separate bar, filled by proportion of concern levels.

Usage

```
create_outbreak_concern(
  long,
  concern_levels = c("Very unconcerned", "Somewhat unconcerned",
    "Neither concerned nor unconcerned", "Somewhat concerned", "Very concerned"),
  outbreak_labels = c("Food/Waterborne Disease Concern", "Food/Waterborne Outbreak Risk",
    "STDs Outbreak Concern", "STDs Transmission Risk",
    "Hemorrhagic Fever Outbreak Concern", "Hemorrhagic Fever Risk", "Zoonoses Concern",
    "Vector-borne Disease Outbreak", "Vector-borne Disease Risk",
    "Airborne Disease Outbreak", "Airborne Disease Risk")
)
```

Arguments

`long` The full long-format tibble from `pivot_disease_data()`.

`concern_levels` Ordered vector of concern levels.

`outbreak_labels` Vector of cleaned outbreak question labels (length = 11).

Value

A ggplot object.

```
create_question_plot_from_long
```

Create a bar chart for a single question across diseases

Description

Visualizes concern levels for a specific question across all diseases. Diseases are ordered left-to-right by proportion of "Very concerned" responses (descending).

Usage

```
create_question_plot_from_long(
  long,
  question_index,
  concern_levels = c("Very unconcerned", "Somewhat unconcerned",
    "Neither concerned nor unconcerned", "Somewhat concerned", "Very concerned"),
  question_labels = c("15m diagnosis delay", "1h diagnosis delay", "4h diagnosis delay",
    "8h diagnosis delay", "1h transport delay", "4h transport delay",
    "8h transport delay", "1 day transport delay", "Patient life impact",
    "Patient wellbeing impact")
)
```

Arguments

`long` The full long-format tibble from `pivot_disease_data()`.
`question_index` Integer from 1 to 10 (position in each block).
`concern_levels` Ordered vector of concern levels.
`question_labels` Vector of question titles (length = 10).

Value

A ggplot object.

export_gpkg	<i>Export data to a GeoPackage</i>
-------------	------------------------------------

Description

Joins input data to country geometries and writes to a GeoPackage file.

Usage

```
export_gpkg(
  data = NULL,
  data_key = "ISO_A3",
  cols = c("overall_risk", "severity_adjusted_risk", data_key),
  .data_path = ""
)
```

Arguments

`data` Data frame with country-level data.
`data_key` Column name for country codes in data.
`cols` Columns to include in the export.
`.data_path` Output file path.

Value

Writes a GeoPackage file to disk.

extract_radar_data	<i>Extract Radar Data for Selected Health Indicators</i>
--------------------	--

Description

Prepares a subset of health indicators for radar chart visualization. The function:

- Aggregates communicable disease indicators into a single score.
- Aggregates injury-related indicators into a single score.
- Combines water and sanitation indicators into a WASH score.
- Includes capacity and vulnerability scores.
- Normalizes all scores to a 0–1 scale by dividing by their column maxima.

Usage

```
extract_radar_data(df)
```

Arguments

df	A data frame with health and risk indicators, including a country column.
----	---

Value

A normalized data frame with radar-specific indicators and a Country column.

geometric_mean	<i>Compute the geometric mean</i>
----------------	-----------------------------------

Description

Calculates the geometric mean of a numeric vector, ignoring NAs.

Usage

```
geometric_mean(x)
```

Arguments

x	Numeric vector.
---	-----------------

Value

Geometric mean as a single numeric value.

import_crpi_excel	<i>Import Excel from previous risk assessment</i>
-------------------	---

Description

Import Excel from previous risk assessment

Usage

```
import_crpi_excel(path)
```

Arguments

path	Path to the Excel file.
------	-------------------------

Value

A tibble with the imported data.

invert_0_10	<i>Invert a 0–10 scale</i>
-------------	----------------------------

Description

Converts values on a 0–10 scale to their inverse ($10 - x$).

Usage

```
invert_0_10(x)
```

Arguments

x	Numeric vector.
---	-----------------

Value

Inverted numeric vector.

invert_0_100	<i>Invert a 0–100 scale</i>
--------------	-----------------------------

Description

Converts values on a 0–100 scale to their inverse (100 - x).

Usage

```
invert_0_100(x)
```

Arguments

x Numeric vector.

Value

Inverted numeric vector.

merge_health_datasets	<i>Merge Global Health and Vulnerability Datasets</i>
-----------------------	---

Description

Merges multiple datasets containing health and risk indicators by ISO3 country codes. The function:

- Removes location_name columns from GBD and HAQ datasets.
- Renames ISO code columns in INFORM and WHO datasets to ISO_A3.
- Merges all datasets using left joins on ISO_A3.
- Resolves duplicate country columns by coalescing them into one.

Usage

```
merge_health_datasets(
  gbd_rates,
  haq_index,
  inform_cap,
  who_indicators,
  inform_severity
)
```

Arguments

gbd_rates	A data frame with GBD rates and ISO_A3 codes.
haq_index	A data frame with HAQ index values and ISO_A3 codes.
inform_cap	A data frame with INFORM indicators and iso3 codes.
who_indicators	A data frame with WHO indicators and iso_a3 codes.
inform_severity	A data frame with INFORM severity index and ISO3 codes.

Value

A merged data frame with all indicators joined by ISO_A3.

normalise_quantiles	<i>Normalize values between the 1st and 99th percentiles</i>
---------------------	--

Description

Scales a numeric vector so that the 1st percentile becomes 0 and the 99th becomes 1.

Usage

```
normalise_quantiles(x)
```

Arguments

x	Numeric vector.
---	-----------------

Value

A normalized numeric vector.

null	<i>Null function</i>
------	----------------------

Description

Use this as template for your first function, and delete it!

Usage

```
null(x = NULL)
```

Arguments

x	(default, NULL)
---	-----------------

Note

You can add all this documentation infrastructure pressing CTRL + SHIFT + ALT + R from anywhere inside the function's body.

Examples

```
## Not run:
  null()
  null(1)

## End(Not run)
```

pivot_disease_data	<i>Pivot the entire survey into long format with disease and question info</i>
--------------------	--

Description

Converts columns 12 to 151 of the survey into long format. Adds disease and question identifiers, calculates proportions.

Usage

```
pivot_disease_data(
  survey,
  concern_levels = c("Very unconcerned", "Somewhat unconcerned",
    "Neither concerned nor unconcerned", "Somewhat concerned", "Very concerned"),
  titles = c("Non-traumatic Chest Pain", "Snakebite",
    "Sudden Febrile illness in malaria-prone area",
    "Sudden Febrile illness in malaria-free area", "Mental Health Crisis",
    "Focal neurological symptoms suggestive of Stroke", "Open Wounds with Hemorrhage",
    "Acute Abdominal Pain", "Lower Back Pain suggestive of renal stones",
    "Head injury or Polytrauma", "Resting dyspnea", "Coma", "Second degree burns",
    "Eye emergency"),
  outbreak_labels = c("Food/Waterborne Outbreak Risk A",
    "Food/Waterborne Outbreak Risk C", "STDs Transmission Risk A",
    "STDs Transmission Risk C", "Hemorrhagic Fever Outbreak Concern A",
    "Hemorrhagic Fever Outbreak Concern C", "Zoonoses Concern A",
    "Vector-borne Disease Risk A", "Vector-borne Disease Risk C",
    "Airborne Disease Risk A", "Airborne Disease Risk C"),
  question_labels = c("15m diagnosis delay", "1h diagnosis delay", "4h diagnosis delay",
    "8h diagnosis delay", "1h transport delay", "4h transport delay",
    "8h transport delay", "1 day transport delay", "Patient life impact",
    "Patient wellbeing impact")
)
```

Arguments

survey A tibble with the full survey (154 columns).
 concern_levels Ordered vector of concern levels (low → high).
 titles A character vector of 14 disease titles.
 outbreak_labels A character vector of outbreak question labels.
 question_labels A character vector of 10 question titles.

Value

A tibble with columns: disease, question, response, prop.

plot_all_diseases_from_long
Plot all diseases using long-format data

Description

Plot all diseases using long-format data

Usage

```
plot_all_diseases_from_long(
  long,
  concern_levels = c("Very unconcerned", "Somewhat unconcerned",
    "Neither concerned nor unconcerned", "Somewhat concerned", "Very concerned")
)
```

Arguments

long The full long-format tibble from pivot_disease_data().
 concern_levels Ordered vector of concern levels.

Value

A named list of ggplot objects.

```
plot_all_questions_from_long
```

Plot all 10 questions across diseases using combined long-format data

Description

Plot all 10 questions across diseases using combined long-format data

Usage

```
plot_all_questions_from_long(
  long,
  concern_levels = c("Very unconcerned", "Somewhat unconcerned",
    "Neither concerned nor unconcerned", "Somewhat concerned", "Very concerned"),
  question_labels = c("15m diagnosis delay", "1h diagnosis delay", "4h diagnosis delay",
    "8h diagnosis delay", "1h transport delay", "4h transport delay",
    "8h transport delay", "1 day transport delay", "Patient life impact",
    "Patient wellbeing impact")
)
```

Arguments

<code>long</code>	A combined tibble from multiple diseases (output of <code>pivot_disease_data()</code>), with an added disease column.
<code>concern_levels</code>	Ordered vector of concern levels.
<code>question_labels</code>	Vector of question titles (length = 10).

Value

A named list of ggplot objects.

```
preprocess_crpi
```

Preprocess CRPI Datasets

Description

Joins two CRPI datasets by ISO A3 code and appends the adjusted index. The function:

- Performs an inner join on `iso_a3`.
- Renames the Index column from the second dataset as `Index_adjusted`.

Usage

```
preprocess_crpi(crpi1, crpi2)
```

Arguments

- | | |
|-------|---|
| crpi1 | A data frame with CRPI data including iso_a3 and Index. |
| crpi2 | A data frame with CRPI data including iso_a3 and Index. |

Value

A merged data frame with original and adjusted CRPI indices.

preprocess_gbd_rates_by_cause

Preprocess GBD DALY Rates by Cause

Description

Processes Global Burden of Disease (GBD) DALY data to extract disease-specific rates. The function:

- Maps GBD location IDs to ISO A3 codes.
- Filters for selected cause IDs and reshapes the data to wide format.
- Computes a composite "Other NCDs" category by summing DALYs for a predefined set of causes.
- Merges the selected causes and the composite category into a single data frame.

Usage

```
preprocess_gbd_rates_by_cause(db = NULL, loc_keys = NULL)
```

Arguments

- | | |
|----------|---|
| db | A data frame with GBD DALY data including location_id, cause_id, and val. |
| loc_keys | A data frame mapping Location ID to ISO_A3 codes. |

Value

A data frame with DALY rates by cause for each country.

```
preprocess_haq_index
```

Preprocess HAQ Index Data

Description

Extracts the most recent Healthcare Access and Quality (HAQ) Index value per country. The function:

- Filters for the HAQ Index indicator.
- Selects the most recent year per country.
- Joins with ISO A3 codes using location metadata.
- Returns a simplified data frame with country name, ISO code, and HAQ index value.

Usage

```
preprocess_haq_index(db = NULL, loc_keys = NULL)
```

Arguments

`db` A data frame with GBD data including `location_id`, `year_id`, and `val`.

`loc_keys` A data frame with `Location ID` and `ISO_A3` columns.

Value

A data frame with HAQ Index values and ISO codes.

```
preprocess_inform
```

Preprocess INFORM Risk and Coping Capacity Data

Description

Merges INFORM Risk and Lack of Coping Capacity (LCC) datasets to extract and transform indicators relevant to vulnerability assessment. The function:

- Extracts numeric indicators from the INFORM Risk dataset.
- Removes metadata rows from the LCC dataset and converts key indicators to numeric.
- Joins both datasets by ISO3 country code.

Usage

```
preprocess_inform(db_risk = NULL, db_lcc = NULL)
```

Arguments

db_risk A data frame with INFORM Risk data.
db_lcc A data frame with Lack of Coping Capacity data.

Value

A merged data frame with risk, vulnerability, and infrastructure indicators.

preprocess_proximity_assessment

Preprocess raw proximity assessment survey data

Description

This function:

- Keeps only the most recent record per country
- Appends ISO_A3 codes from risk_country
- Standardizes diagnosis and transport delay values
- Dynamically renames outbreak concern columns based on response content
- Removes metadata and free-text columns not needed for scoring
- Renames the timestamp column to "time"

Usage

```
preprocess_proximity_assessment(form, risk_country)
```

Arguments

form A tibble containing raw proximity assessment data
risk_country A tibble with columns country and ISO_A3

Value

A cleaned tibble ready for scoring

preprocess_severity	<i>Preprocess INFORM Severity Index Data</i>
---------------------	--

Description

Cleans and transforms INFORM Severity Index data for humanitarian crises. The function:

- Removes the first row and columns with only NA values.
- One-hot encodes the DRIVERS column by splitting comma-separated values.
- Renames key columns for clarity.
- Computes a new variable conditions_impact as the geometric mean of people_conditions and crisis_impact.

Usage

```
preprocess_severity(df)
```

Arguments

df	A data frame containing INFORM Severity Index data.
----	---

Value

A processed data frame with renamed columns, one-hot encoded drivers, and derived indicators.

preprocess_survey	<i>Preprocess the survey dataset</i>
-------------------	--------------------------------------

Description

This function:

- standardizes column names to snake_case
- removes metadata columns from ID to "Which is your role"
- converts all remaining columns to factors with a predefined level order

Usage

```
preprocess_survey(data)
```

Arguments

data	A tibble or data frame containing the raw survey data.
------	--

Value

A cleaned tibble with standardized variable names and factor columns.

Examples

```
# Example survey data
survey <- tibble::tibble(
  disease = c("Acute Abdominal Pain", "Acute Abdominal Pain"),
  question = c("4h diagnosis delay", "4h transport delay"),
  response = c("Very concerned", "Somewhat concerned"),
  n = c(25, 30),
  prop = c(0.35, 0.45)
)

cleaned <- preprocess_survey(survey)
```

preprocess_u5mr

*Preprocess Under-5 Mortality Rate Data***Description**

Filters and processes under-five mortality rate data from the UN IGME database. The function:

- Filters for total population, valid observation status, and relevant wealth quintiles.
- Removes unnecessary metadata columns.
- Selects the most recent observation per country and wealth quintile.
- Calculates the difference in mortality between the highest and lowest wealth quintiles.
- Normalizes this disparity using quantile normalization.
- Filters out aggregate or non-country entries based on REF_AREA codes.

Usage

```
preprocess_u5mr(db = NULL)
```

Arguments

db A data frame containing UN IGME under-five mortality rate data.

Value

A data frame with country-level under-five mortality disparities and normalized indicators.

preprocess_who_data	<i>Preprocess WHO Health Indicator Data</i>
---------------------	---

Description

Filters and joins multiple WHO datasets to extract the most recent values for selected indicators. The function:

- Filters each dataset for country-level data and the most recent year.
- Applies additional filters for urbanization, sex, and AMR awareness where applicable.
- Renames each indicator column using the indicator name.
- Joins all datasets by country name.
- Optionally joins ISO A3 codes using a location key table.
- Returns a tidy data frame with one row per country and one column per indicator.

Usage

```
preprocess_who_data(  
  relay_may2023_wide = NULL,  
  x9a706fd_all_latest = NULL,  
  x19e688d_all_latest = NULL,  
  x217795a_all_latest = NULL,  
  b9c6c79_all_latest = NULL,  
  bbf3a64_all_latest = NULL,  
  d2a45a5_all_latest = NULL,  
  ed50112_all_latest = NULL,  
  who_location_keys = NULL  
)
```

Arguments

relay_may2023_wide	WHO dataset on household health expenditure.
x9a706fd_all_latest	WHO dataset on UHC coverage index.
x19e688d_all_latest	WHO dataset on antibiotic consumption.
x217795a_all_latest	WHO dataset on doctor density.
b9c6c79_all_latest	WHO dataset on government health expenditure.
bbf3a64_all_latest	WHO dataset on ODA to health.
d2a45a5_all_latest	WHO dataset on access to medicines.

ed50112_all_latest

WHO dataset on unsafe WASH deaths.

who_location_keys

Optional data frame with GEO_NAME_SHORT and ISO_A3 for country code mapping.

Value

A data frame with selected WHO health indicators by country.

process_ami_survey	<i>Process AMI survey responses enforcing monotonic concern by group</i>
--------------------	--

Description

Enforces non-decreasing concern within two groups: (1) diagnosis delay (first 4 columns) and (2) transport delay (next 4 columns). The last two columns (health/wellbeing) are independent.

Within each of the first two groups and within each row, scanning left-to-right, any entry lower than the highest seen so far is raised to that highest. NAs are treated the same (filled with the highest seen so far). Leading NAs remain NA until a level appears later in that group.

"Not relevant for this condition" (case-insensitive) is converted to NA by default. All columns are returned as ordered factors with the same concern_levels.

Usage

```
process_ami_survey(
  data,
  diag_cols = 1:4,
  transport_cols = 5:8,
  impact_cols = 9:10,
  concern_levels = c("Very unconcerned", "Somewhat unconcerned",
    "Neither concerned nor unconcerned", "Somewhat concerned", "Very concerned"),
  treat_not_relevant_as_na = TRUE
)
```

Arguments

data	A data frame containing the 10 AMI columns (in any order).
diag_cols	Columns (names or positions) for the diagnosis delay group (length 4). Default: first 4 columns.
transport_cols	Columns (names or positions) for the transport delay group (length 4). Default: next 4 columns.
impact_cols	Columns (names or positions) for health/wellbeing (length 2). Default: last 2 columns.
concern_levels	Character vector of ordered concern levels (low → high).
treat_not_relevant_as_na	Logical; turn strings with "Not relevant" into NA (default TRUE).

Value

A data frame with the same columns, all as ordered factors with uniform levels, and with monotonic constraints enforced within the first two groups.

run_app	<i>Launch the Country Risk Explorer Shiny Application</i>
---------	---

Description

This function initializes and launches an interactive Shiny web application designed to explore and visualize country-level health and crisis risk data. The app includes multiple interactive visualizations such as a lollipop plot, 3D scatterplot, radar chart, risk distribution histogram, and an adjusted risk map.

Usage

```
run_app(raw_data, radar_data, corrected_radar_data)
```

Arguments

raw_data	A data frame containing the full dataset with country-level health and risk indicators. This dataset is used across all visualizations.
radar_data	A data frame containing a subset of the raw data specifically formatted for radar chart visualization. It should include numeric indicators relevant to health risk dimensions.
corrected_radar_data	A data frame similar in structure to radar_data, but with severity-adjusted values used to compare original and corrected risk profiles.

Value

This function does not return a value. It launches a Shiny application in the user's default web browser.

summarise_outbreak_data	<i>Summarise outbreak concern proportions with counts</i>
-------------------------	---

Description

- This function:
- Computes the number and proportion of "Very concerned" responses per column
 - Splits column names into disease and response type
 - Relabels type A as "Very concerned" and type C as "Confirmed"
 - Ensures Zoonoses - Confirmed matches Zoonoses - Very concerned

Usage

```
summarise_outbreak_data(outbreak_block)
```

Arguments

outbreak_block A tibble with multiple outbreak concern columns

Value

A tibble with columns: disease, question, response, n, prop

update_risk_with_cm	<i>Adjust Radar Data Using Crisis Modifier Drivers</i>
---------------------	--

Description

Updates radar chart indicators by adjusting them based on crisis drivers and severity. The function:

- Selects relevant columns from the risk score data.
- Computes a CoPA-IotC score from people conditions and crisis impact.
- Replaces missing values with defaults.
- Renames driver columns and ensures all logical indicators from cm_data are present.
- For each driver, increments logical indicators based on presence.
- Applies the update_score() function to adjust radar indicators using CoPA-IotC and driver counts.

Usage

```
update_risk_with_cm(cm_data, risk_score, radar_data)
```

Arguments

cm_data	A data frame with driver names and associated logical indicators.
risk_score	A data frame with country-level risk and driver data.
radar_data	A data frame with original radar scores.

Value

A data frame with adjusted radar scores for each country.

`update_score`*Update a Score Based on Severity Multiplier and Repetition Count*

Description

Adjusts a base score using a severity multiplier. The adjustment is applied multiple times, capped at 3 repetitions. Each iteration increases the score proportionally to the severity.

Usage

```
update_score(base_value, multiplier, times = 1)
```

Arguments

<code>base_value</code>	A numeric vector of base scores.
<code>multiplier</code>	A numeric vector of severity multipliers.
<code>times</code>	An integer vector indicating how many times to apply the adjustment (default is 1, capped at 3).

Value

A numeric vector of adjusted scores.

Examples

```
update_score(base_value = c(0.6, 0.7), multiplier = c(3, NA), times = c(2, 1))
```

Index

`add_capacity_score`, [4](#)
`add_hazard_score`, [4](#)
`add_localised_risk`, [5](#)
`add_overall_risk`, [6](#)
`add_proximity_score`, [6](#)
`add_severity`, [7](#)
`add_vulnerability_score`, [8](#)

`create_3d_plot`, [18](#)
`create_correlation_matrix_with_groups`,
 [19](#)
`create_disease_plot_from_long`, [20](#)
`create_disease_scatter`, [20](#)
`create_lollipop`, [21](#)
`create_lollipop_plot_comp`, [22](#)
`create_lollipop_plot_shift`, [23](#)
`create_outbreak_concern`, [23](#)
`create_question_plot_from_long`, [24](#)
`createimg_bars`, [9](#)
`createimg_cap_bars`, [9](#)
`createimg_daly_bars`, [10](#)
`createimg_risk_bars`, [10](#)
`createimg_risk_hist`, [11](#)
`createimg_u5mr_bars`, [11](#)
`createimg_vul_bars`, [12](#)
`createmap`, [12](#)
`createmap_adjusted_risk_score`, [13](#)
`createmap_cap_avg`, [14](#)
`createmap_cap_score`, [14](#)
`createmap_DALY_avg`, [15](#)
`createmap_DALY_score`, [15](#)
`createmap_risk_score`, [16](#)
`createmap_severity_score`, [16](#)
`createmap_u5mr_avg`, [17](#)
`createmap_u5mr_did`, [17](#)
`createmap_vul_score`, [18](#)

`export_gpkg`, [25](#)
`extract_radar_data`, [26](#)
`geometric_mean`, [26](#)
`import_crpi_excel`, [27](#)
`invert_0_10`, [27](#)
`invert_0_100`, [28](#)
`merge_health_datasets`, [28](#)
`normalise_quantiles`, [29](#)
`null`, [29](#)
`pivot_disease_data`, [30](#)
`plot_all_diseases_from_long`, [31](#)
`plot_all_questions_from_long`, [32](#)
`preprocess_crpi`, [32](#)
`preprocess_gbd_rates_by_cause`, [33](#)
`preprocess_haq_index`, [34](#)
`preprocess_inform`, [34](#)
`preprocess_proximity_assessment`, [35](#)
`preprocess_severity`, [36](#)
`preprocess_survey`, [36](#)
`preprocess_u5mr`, [37](#)
`preprocess_who_data`, [38](#)
`process_ami_survey`, [39](#)
`run_app`, [40](#)
`summarise_outbreak_data`, [40](#)
`update_risk_with_cm`, [41](#)
`update_score`, [42](#)